FF Heuristic: Complete Guide

Classical Planning and Delete Relaxation

Machine Intelligence - Lecture 4
Comprehensive Cheat Sheet

A Detailed Exploration of the FF Heuristic Algorithm

Including Step-by-Step Examples, Properties, and Comparisons

Contents

1 Introduction to FF Heuristic **3**
- 1.1 Overview . . . 3
- 1.2 Key Concept: Delete Relaxation . . . 3
- 1.3 Why FF Heuristic? . . . 3
- 1.4 Notation Review . . . 3
- 1.5 The h+ Heuristic . . . 3
- 1.6 FF vs h+ . . . 3

2 The FF Algorithm **3**
- 2.1 High-Level Overview . . . 3
- 2.2 Complete Algorithm Pseudocode . . . 4
- 2.3 Detailed Line-by-Line Explanation . . . 5
- 2.4 Key Algorithmic Insights . . . 5

3 Detailed Example 1: Transport Problem **5**
- 3.1 Problem Definition . . . 5
- 3.2 Phase 1: Forward Layer Construction . . . 6
- 3.3 Recording Levels . . . 6
- 3.4 Goal Distribution . . . 6
- 3.5 Phase 2: Backward Goal Extraction . . . 7
- 3.6 Final Result . . . 7

4 Detailed Example 2: City Traversal **7**
- 4.1 Problem Definition . . . 7
- 4.2 Phase 1: Forward Layer Construction . . . 8
- 4.3 Recording Levels . . . 8
- 4.4 Goal Distribution . . . 8
- 4.5 Phase 2: Backward Goal Extraction . . . 8
- 4.6 Final Result . . . 9

5 Why FF is Not Admissible **9**
- 5.1 Understanding Inadmissibility . . . 9
- 5.2 Conceptual Explanation . . . 9
- 5.3 Why Use FF Then? . . . 9

6 Alternative Delete-Relaxation Heuristics **10**
- 6.1 Overview . . . 10
- 6.2 The hmax Heuristic . . . 10
- 6.3 The hadd Heuristic . . . 10
- 6.4 Comparison: FF vs hadd . . . 10
- 6.5 The LM-Cut Heuristic . . . 10
- 6.6 Summary Table . . . 10

7 Practical Considerations **11**
- 7.1 When to Use FF . . . 11
- 7.2 Computational Complexity . . . 11
- 7.3 Implementation Tips . . . 11
- 7.4 Common Mistakes . . . 11

8 Advanced Topics **11**
- 8.1 FF Planner . . . 11
- 8.2 Helpful Actions . . . 11

8.3 Variations of FF . . . 11
8.4 Integration with Search . . . 12

9 Connections to Other Concepts **12**
9.1 Planning Graphs . . . 12
9.2 Relationship to h+ and h* . . . 12
9.3 Delete Relaxation in General . . . 12
9.4 Domain-Independent Planning . . . 12

10 Worked Practice Problems **12**
10.1 Practice Problem 1: Simple Blocks World . . . 12
10.2 Practice Problem 2: Simple Grid World . . . 12

11 Key Takeaways **13**

12 Further Reading **13**

13 Exam Preparation **13**
13.1 Common Exam Questions . . . 13
13.2 Quick Reference . . . 13

14 Conclusion **13**

1 Introduction to FF Heuristic

1.1 Overview

The **FF (Fast Forward) heuristic** is a delete-relaxation heuristic used in classical planning. It computes an estimate of the cost to reach the goal from a given state by solving a relaxed version of the planning problem.

1.2 Key Concept: Delete Relaxation

Delete Relaxation ignores the delete effects of actions:

- Original action: a with preconditions pre_a and effects $\text{eff}_a = \text{eff}_a^+ \cup \text{eff}_a^-$
- Relaxed action: a^+ with preconditions pre_a and effects only eff_a^+ (positive effects)
- This makes the problem easier since facts once achieved remain true

1.3 Why FF Heuristic?

Motivation

- Provides informative heuristic values
- Computationally efficient
- Works well in practice for many planning domains
- Balances accuracy and computation time

1.4 Notation Review

Planning Task:

- $\mathcal{F}$: Set of facts (propositions)
- $s \subseteq \mathcal{F}$: A state
- $\mathcal{A}$: Set of actions
- $\text{pre}_a \subseteq \mathcal{F}$: Preconditions of action a
- $\text{eff}_a \subseteq \mathcal{F}$: Effects of action a
- I: Initial state
- $G \subseteq \mathcal{F}$: Goal conditions

1.5 The h+ Heuristic

Before understanding FF, we need to understand h^+:

h+ Definition

$h^+(s)$ = cost of optimal relaxed plan from state s
This is the **optimal** solution to the delete-relaxed problem.

Properties of h+:

- Admissible: $h^+(s) \leq h^*(s)$ where $h^*(s)$ is optimal cost
- Provides perfect heuristic for relaxed problem
- NP-hard to compute exactly

1.6 FF vs h+

Property	**h+**	**FF**
Optimality	Optimal relaxed	Suboptimal relaxed
Admissibility	Yes	No
Computation	NP-hard	Polynomial
Practical use	Too slow	Fast, practical

FF approximates h^+ by finding *some* relaxed plan, not necessarily optimal.

2 The FF Algorithm

2.1 High-Level Overview

The FF heuristic works in **two main phases**:

Two Phases

Phase 1: Forward Layer Construction

- Build fact layers $F_0, F_1, F_2, \ldots$ forward
- Build action layers $A_0, A_1, A_2, \ldots$ forward
- Continue until all goals are achieved

Phase 2: Backward Goal Extraction

- Start from the goal
- Work backwards through layers
- Select supporting actions for each subgoal
- Sum costs of selected actions

2.2 Complete Algorithm Pseudocode

Algorithm 1 FF Heuristic

```
function FF(state s)
    F_0 ← s                                                  ▷ Initialize first fact layer
    m ← 0                                                    ▷ Layer counter
    while G ⊈ F_m do                                         ▷ Until goals achieved
        A_m ← {a ∈ 𝒜 | pre_a ⊆ F_m}
        F_{m+1} ← F_m ∪ ⋃_{a∈A_m} eff_a
        if F_m = F_{m+1} then
            return ∞                                         ▷ Goals unreachable
        end if
        m ← m + 1
    end while
    for each f ∈ F_m do
        level(f) = min{t | f ∈ F_t}
    end for
    for each a ∈ A_{m−1} do
        level(a) = min{t | a ∈ A_t}
    end for
    for t = 1, ..., m do
        G_t ← {g ∈ G | level(g) = t}
    end for
    C ← 0                                                    ▷ Cost accumulator
    for t = m, ..., 1 do
        for each g ∈ G_t do
            Select any a ∈ 𝒜 s.t. level(a) = t − 1 and g ∈ eff_a
            C ← C + cost(a)
            for each f ∈ pre_a do
                G_{level(f)} ← G_{level(f)} ∪ {f}
            end for
        end for
    end for
    return C
end function
```

2.3 Detailed Line-by-Line Explanation

Lines 1-2: Initialization

$F_0 \leftarrow s;\ m \leftarrow 0$
Initialize the first fact layer with the current state and set layer counter to 0.

Lines 3-9: Forward Pass - Building Layers

`while` $G \not\subseteq F_m$ `do`
Continue building layers until all goal facts are present in some fact layer.

Line 4: Applicable Actions

$A_m \leftarrow \{a \in \mathcal{A} \mid \texttt{pre}_a \subseteq F_m\}$
Collect all actions whose preconditions are satisfied by facts in layer F_m. These actions are "applicable" at layer m.

Line 5: Next Fact Layer

$F_{m+1} \leftarrow F_m \cup \bigcup_{a \in A_m} \texttt{eff}_a$
The next fact layer contains:

- All facts from previous layer F_m (persistence)
- All effects from applicable actions in A_m

Note: In delete relaxation, we never remove facts!

Lines 6-8: Fixpoint Check

`if` $F_m = F_{m+1}$ `then return` ∞
If no new facts are added, we've reached a fixpoint. If goals are still not achieved, they're unreachable. Return infinity to indicate this.

Lines 10-12: Fact Levels

$\texttt{level}(f) = \min\{t \mid f \in F_t\}$
For each fact, record the *earliest* layer at which it appears. This is crucial for the backward pass.

Lines 13-15: Action Levels

$\texttt{level}(a) = \min\{t \mid a \in A_t\}$
For each action, record the *earliest* layer at which it becomes applicable.

Lines 16-18: Goal Distribution

$G_t \leftarrow \{g \in G \mid \texttt{level}(g) = t\}$
Distribute goal facts into sets based on when they first appear. This organizes the backward search.

Lines 19-28: Backward Pass - Extracting Plan

The Backward Pass Strategy:
Starting from the highest layer, work backwards:

1. For each goal at layer t
2. Select a supporting action from layer $t-1$
3. Add action's cost to total
4. Add action's preconditions as new subgoals

Line 22: Action Selection

`Select any` a `s.t.` $\texttt{level}(a) = t-1$ and $g \in \text{eff}_a$
Choose an action from the previous layer that achieves the current goal g. The choice is arbitrary (any such action works).

Line 23: Cost Accumulation

$C \leftarrow C + \texttt{cost}(a)$
Add the cost of the selected action to the total heuristic value.

Lines 24-26: Regression

$G_{\texttt{level}(f)} \leftarrow G_{\texttt{level}(f)} \cup \{f\}$ `for each` $f \in \texttt{pre}_a$
Add each precondition of the selected action as a subgoal at its appropriate level. This is the "regression" step.

2.4 Key Algorithmic Insights

Important Points

1. **Fact Persistence**: In delete relaxation, once a fact is true, it remains true in all subsequent layers.
2. **Layer Monotonicity**: $F_0 \subseteq F_1 \subseteq F_2 \subseteq \dots$
3. **Action Selection**: The choice of which action to select in the backward pass is arbitrary. Different choices yield different (but valid) relaxed plans.
4. **Subgoal Organization**: Using $\text{level}(f)$ ensures we request each fact at the earliest layer it appears.
5. **Termination**: The forward pass terminates either when goals are achieved or a fixpoint is reached.

3 Detailed Example 1: Transport Problem

3.1 Problem Definition

Domain: Simple transport domain with a truck and package.
Variables:

- t: truck location $\in \{A, B, C, D\}$
- p: package location $\in \{A, B, C, D, t\}$ where t means "in truck"

Initial State: $I = \{t = A, p = C\}$
Goal: $G = \{t = D, p = D\}$
Actions:

- drive(x, y): pre: $t = x$; eff: $t = y$
- load(x): pre: $t = x, p = x$; eff: $p = t$
- unload(x): pre: $t = x, p = t$; eff: $p = x$

All actions have unit cost.

3.2 Phase 1: Forward Layer Construction

Layer 0: Initial State

F0

$F_0 = \{t = A, p = C\}$
This is our starting point.

Finding Applicable Actions at Layer 0:

Check each action:

- drive(A, B): pre $= \{t = A\}$, $\{t = A\} \subseteq F_0$? YES
- drive(A, C): pre $= \{t = A\}$, $\{t = A\} \subseteq F_0$? YES
- drive(A, D): pre $= \{t = A\}$, $\{t = A\} \subseteq F_0$? YES
- load(A): pre $= \{t = A, p = A\}$, $\{t = A, p = A\} \subseteq F_0$? NO ($p = A$ not in F_0)
- load(C): pre $= \{t = C, p = C\}$, $\{t = C, p = C\} \subseteq F_0$? NO ($t = C$ not in F_0)

A0

$A_0 = \{\text{drive}(A, B), \text{drive}(A, C), \text{drive}(A, D)\}$
But for simplicity, consider $A_0 = \{\text{drive}(A, B)\}$ as shown in lecture.

Layer 1:

F1

$F_1 = F_0 \cup \{\text{eff of drive}(A, B)\}$
$= \{t = A, p = C\} \cup \{t = B\}$
$= \{t = A, p = C, t = B\}$
Note: In delete relaxation, $t = A$ persists even though truck moved to B!

Finding Applicable Actions at Layer 1:

Now with $F_1 = \{t = A, p = C, t = B\}$:

- drive(B, C): pre $= \{t = B\}$, applicable? YES
- drive(B, A): pre $= \{t = B\}$, applicable? YES
- Other drives from A: also applicable (but already included)

A1

$A_1 = \{\text{drive}(B, C), \text{drive}(B, A)\}$
Following lecture, we focus on drive(B, C).

Layer 2:

F2

$F_2 = F_1 \cup \{\text{eff of drive}(B, C)\}$
$= \{t = A, p = C, t = B\} \cup \{t = C\}$
$= \{t = A, p = C, t = B, t = C\}$

Finding Applicable Actions at Layer 2:

Now with $F_2 = \{t = A, p = C, t = B, t = C\}$:

- load(C): pre $= \{t = C, p = C\}$, both in F_2? YES
- drive(C, D): pre $= \{t = C\}$, applicable? YES
- drive(C, B): pre $= \{t = C\}$, applicable? YES

A2

$A_2 = \{\text{drive}(C, D), \text{drive}(C, B), \text{load}(C)\}$

Layer 3:

F3

$F_3 = F_2 \cup \{\text{eff of actions in } A_2\}$
Effects from A_2:

- drive(C, D) adds $t = D$
- load(C) adds $p = t$

$F_3 = \{t = A, p = C, t = B, t = C, t = D, p = t\}$

Finding Applicable Actions at Layer 3:

- unload(D): pre $= \{t = D, p = t\}$, both in F_3? YES
- unload(A): pre $= \{t = A, p = t\}$, both in F_3? YES
- Similarly for unload at B, C

A3

$A_3 = \{\text{unload}(A), \text{unload}(B), \text{unload}(C), \text{unload}(D)\}$

Layer 4:

F4

$F_4 = F_3 \cup \{\text{effects from } A_3\}$
Unload actions add $p = A, p = B, p = C, p = D$
$F_4 = \{t = A, p = C, t = B, t = C, t = D, p = t, p = A, p = B, p = D\}$
Check goal: $G = \{t = D, p = D\}$
Is $G \subseteq F_4$? YES! Both $t = D$ and $p = D$ are in F_4.

Forward pass terminates at layer 4.

3.3 Recording Levels

Now we record when each fact and action first appeared:

Fact Levels:

- level$(t = A) = 0$
- level$(p = C) = 0$
- level$(t = B) = 1$
- level$(t = C) = 2$
- level$(t = D) = 3$
- level$(p = t) = 3$
- level$(p = A) = 4$
- level$(p = B) = 4$
- level$(p = D) = 4$

Action Levels:

- level(drive$(A, B)) = 0$
- level(drive$(B, C)) = 1$
- level(drive$(C, D)) = 2$
- level(load$(C)) = 2$
- level(unload$(D)) = 3$

3.4 Goal Distribution

Distribute goals by their levels:

- $G_0 = \{\}$ (no goals at level 0)
- $G_1 = \{\}$
- $G_2 = \{\}$
- $G_3 = \{t = D\}$ (level$(t = D) = 3$)
- $G_4 = \{p = D\}$ (level$(p = D) = 4$)

3.5 Phase 2: Backward Goal Extraction

Processing Layer 4 Goals:

Layer 4: Goal $p = D$

Need to achieve $p = D$ at layer 4.
Find action a where:
- $\text{level}(a) = 3$ (one layer before)
- $p = D \in \text{eff}_a$

Action $\text{unload}(D)$ satisfies this:
- $\text{level}(\text{unload}(D)) = 3$ ✓
- $\text{eff}(\text{unload}(D)) = \{p = D\}$ ✓

Select: $\text{unload}(D)$
Cost: $C = 0 + 1 = 1$
Preconditions: $\text{pre}(\text{unload}(D)) = \{t = D, p = t\}$
Add these as subgoals:
- $G_3 = G_3 \cup \{t = D, p = t\} = \{t = D, p = t\}$ (already had $t = D$)

Processing Layer 3 Goals:

Layer 3: Goal $t = D$

Need action at layer 2 that achieves $t = D$.
Action $\text{drive}(C, D)$:
- $\text{level}(\text{drive}(C, D)) = 2$ ✓
- $\text{eff}(\text{drive}(C, D)) = \{t = D\}$ ✓

Select: $\text{drive}(C, D)$
Cost: $C = 1 + 1 = 2$
Preconditions: $\text{pre}(\text{drive}(C, D)) = \{t = C\}$
Add as subgoal: $G_2 = G_2 \cup \{t = C\} = \{t = C\}$

Layer 3: Goal $p = t$

Need action at layer 2 that achieves $p = t$.
Action $\text{load}(C)$:
- $\text{level}(\text{load}(C)) = 2$ ✓
- $\text{eff}(\text{load}(C)) = \{p = t\}$ ✓

Select: $\text{load}(C)$
Cost: $C = 2 + 1 = 3$
Preconditions: $\text{pre}(\text{load}(C)) = \{t = C, p = C\}$
Add as subgoals: $G_2 = G_2 \cup \{t = C, p = C\} = \{t = C, p = C\}$
(Note: $t = C$ already there)

Processing Layer 2 Goals:

Layer 2: Goal $t = C$

Need action at layer 1 that achieves $t = C$.
Action $\text{drive}(B, C)$:
- $\text{level}(\text{drive}(B, C)) = 1$ ✓
- $\text{eff}(\text{drive}(B, C)) = \{t = C\}$ ✓

Select: $\text{drive}(B, C)$
Cost: $C = 3 + 1 = 4$
Preconditions: $\text{pre}(\text{drive}(B, C)) = \{t = B\}$
Add as subgoal: $G_1 = G_1 \cup \{t = B\} = \{t = B\}$

Layer 2: Goal $p = C$

Check: $\text{level}(p = C) = 0$
This fact is in the initial state, so no action needed to support it at layer 2. It persists from F_0.

Processing Layer 1 Goals:

Layer 1: Goal $t = B$

Need action at layer 0 that achieves $t = B$.
Action $\text{drive}(A, B)$:
- $\text{level}(\text{drive}(A, B)) = 0$ ✓
- $\text{eff}(\text{drive}(A, B)) = \{t = B\}$ ✓

Select: $\text{drive}(A, B)$
Cost: $C = 4 + 1 = 5$
Preconditions: $\text{pre}(\text{drive}(A, B)) = \{t = A\}$
Check: $\text{level}(t = A) = 0$, in initial state. Done!

3.6 Final Result

FF Heuristic Value

$h_{FF}(s) = 5$
The relaxed plan found:
1. $\text{drive}(A, B)$ - cost 1
2. $\text{drive}(B, C)$ - cost 1
3. $\text{load}(C)$ - cost 1
4. $\text{drive}(C, D)$ - cost 1
5. $\text{unload}(D)$ - cost 1

Total cost: 5

Comparison with h+:

In this example, $h^+(s) = 5$ as well. The FF heuristic found an optimal relaxed plan! This shows that FF can sometimes equal h^+.

Note: The optimal real (unrelaxed) plan has the same cost of 5, so this state actually has $h^*(s) = 5$. The delete relaxation didn't help much here because there aren't many delete effects that matter.

4 Detailed Example 2: City Traversal

4.1 Problem Definition

Domain: Traveling salesman variant - visit all cities and return home.

Variables:
- $\text{at} \in \{\text{Sy}, \text{Ad}, \text{Br}, \text{Pe}, \text{Da}\}$ (Sydney, Adelaide, Brisbane, Perth, Darwin)
- $v(x) \in \{T, F\}$ for each city x (whether city has been visited)

Initial State:
- $I = \{\text{at} = \text{Sy}, v(\text{Sy}) = T, v(\text{Ad}) = F, v(\text{Br}) = F, v(\text{Pe}) = F, v(\text{Da}) = F\}$

Goal:
- $G = \{\text{at} = \text{Sy}, v(\text{Sy}) = T, v(\text{Ad}) = T, v(\text{Br}) = T, v(\text{Pe}) = T, v(\text{Da}) = T\}$

Visit all cities and return to Sydney.

Actions: $\text{drive}(x, y)$ for each pair of cities
- Precondition: $\text{at} = x$
- Effect: $\text{at} = y, v(y) = T$

Action Costs:
- $\text{cost}(\text{drive}(\text{Sy}, \text{Br})) = 1$
- $\text{cost}(\text{drive}(\text{Sy}, \text{Ad})) = 2$
- $\text{cost}(\text{drive}(\text{Ad}, \text{Pe})) = 3$
- $\text{cost}(\text{drive}(\text{Ad}, \text{Da})) = 4$
- Other costs not specified (assume unit cost if needed)

4.2 Phase 1: Forward Layer Construction

Layer 0: Initial State

F0

$F_0 = \{\text{at} = \text{Sy}, v(\text{Sy}) = T\}$
(We omit the $v(x) = F$ facts for brevity)

Applicable Actions at Layer 0:

From Sydney, we can drive to any other city:

- drive(Sy, Br): pre = {at = Sy} ✓
- drive(Sy, Ad): pre = {at = Sy} ✓
- drive(Sy, Pe): pre = {at = Sy} ✓
- drive(Sy, Da): pre = {at = Sy} ✓

A0

$A_0 = \{\text{drive}(\text{Sy}, \text{Br}), \text{drive}(\text{Sy}, \text{Ad}), \ldots\}$
Per lecture slides, we focus on: $A_0 = \{\text{drive}(\text{Sy}, \text{Br}), \text{drive}(\text{Sy}, \text{Ad})\}$

Layer 1:

F1

$F_1 = F_0 \cup \{\text{effects from } A_0\}$
Effects:

- drive(Sy, Br) adds: at = Br, v(Br) = T
- drive(Sy, Ad) adds: at = Ad, v(Ad) = T

$F_1 = \{\text{at} = \text{Sy}, v(\text{Sy}) = T, \text{at} = \text{Br}, v(\text{Br}) = T, \text{at} = \text{Ad}, v(\text{Ad}) = T\}$
Note: In relaxation, at = Sy persists alongside at = Br and at = Ad!

Applicable Actions at Layer 1:

Now we can drive from Brisbane or Adelaide:

From Adelaide:

- drive(Ad, Pe): pre = {at = Ad}, at = Ad $\in F_1$ ✓
- drive(Ad, Da): pre = {at = Ad} ✓
- drive(Ad, Sy): pre = {at = Ad} ✓

From Brisbane:

- drive(Br, Sy): pre = {at = Br} ✓

A1

$A_1 = \{\text{drive}(\text{Ad}, \text{Pe}), \text{drive}(\text{Ad}, \text{Da}), \text{drive}(\text{Ad}, \text{Sy}), \text{drive}(\text{Br}, \text{S}$

Layer 2:

F2

$F_2 = F_1 \cup \{\text{effects from } A_1\}$
New effects:

- at = Pe, v(Pe) = T
- at = Da, v(Da) = T

$F_2 = F_1 \cup \{\text{at} = \text{Pe}, v(\text{Pe}) = T, \text{at} = \text{Da}, v(\text{Da}) = T\}$
Now we have all facts including:

- at can be any city (Sy, Ad, Br, Pe, Da)
- All v(city) = T

Check goal: $G = \{\text{at} = \text{Sy}, v(\text{Sy}) = T, v(\text{Ad}) = T, v(\text{Br}) = T, v(\text{Pe}) = T, v(\text{Da}) = T\}$
All goal facts are in F_2! Forward pass complete.

4.3 Recording Levels

Fact Levels:

- level($at = Sy$) = 0
- level($v(Sy) = T$) = 0
- level($at = Br$) = 1
- level($v(Br) = T$) = 1
- level($at = Ad$) = 1
- level($v(Ad) = T$) = 1
- level($at = Pe$) = 2
- level($v(Pe) = T$) = 2
- level($at = Da$) = 2
- level($v(Da) = T$) = 2

Action Levels:

- level(drive(Sy, Br)) = 0
- level(drive(Sy, Ad)) = 0
- level(drive(Ad, Pe)) = 1
- level(drive(Ad, Da)) = 1
- level(drive(Ad, Sy)) = 1
- level(drive(Br, Sy)) = 1

4.4 Goal Distribution

- $G_0 = \{v(\text{Sy}) = T\}$ (already in initial state)
- $G_1 = \{v(\text{Br}) = T, v(\text{Ad}) = T\}$
- $G_2 = \{\text{at} = \text{Sy}, v(\text{Pe}) = T, v(\text{Da}) = T\}$
  Wait, level($at = Sy$) = 0, not 2. Let me recalculate:
  Actually, level($at = Sy$) = 0 since it's in initial state.
  Correct distribution:
- $G_0 = \{\text{at} = \text{Sy}, v(\text{Sy}) = T\}$
- $G_1 = \{v(\text{Br}) = T, v(\text{Ad}) = T\}$
- $G_2 = \{v(\text{Pe}) = T, v(\text{Da}) = T\}$

But we need to return to Sydney, and at = Sy is already at level 0. The goal includes it, but it's already achieved from the start (and persists in relaxation).

4.5 Phase 2: Backward Goal Extraction

Processing Layer 2 Goals:

Layer 2: Goal $v(\text{Pe}) = T$

Need action at layer 1 achieving $v(\text{Pe}) = T$.
Action drive(Ad, Pe):

- level(drive(Ad, Pe)) = 1 ✓
- $v(\text{Pe}) = T \in$ eff(drive(Ad, Pe)) ✓

Select: drive(Ad, Pe)
Cost: $C = 0 + 3 = 3$
Preconditions: {at = Ad}
Add subgoal: $G_1 = G_1 \cup \{\text{at} = \text{Ad}\}$

Layer 2: Goal $v(\text{Da}) = T$

Need action at layer 1 achieving $v(\text{Da}) = T$.
Action drive(Ad, Da):

- level(drive(Ad, Da)) = 1 ✓
- $v(\text{Da}) = T \in$ eff(drive(Ad, Da)) ✓

Select: drive(Ad, Da)
Cost: $C = 3 + 4 = 7$
Preconditions: {at = Ad}
Add subgoal: $G_1 = G_1 \cup \{\text{at} = \text{Ad}\}$ (already there)

Processing Layer 1 Goals:

Current $G_1 = \{v(\text{Br}) = T, v(\text{Ad}) = T, \text{at} = \text{Ad}\}$

Layer 1: Goal $v(\text{Br}) = T$

Need action at layer 0 achieving $v(\text{Br}) = T$.
Action drive(Sy, Br):
- level(drive(Sy, Br)) = 0 ✓
- $v(\text{Br}) = T \in \text{eff}$ ✓

Select: drive(Sy, Br)
Cost: $C = 7 + 1 = 8$
Preconditions: {at = Sy}
This is in initial state (level 0), so we're done with this branch.

Layer 1: Goal $v(\text{Ad}) = T$

Need action at layer 0 achieving $v(\text{Ad}) = T$.
Action drive(Sy, Ad):
- level(drive(Sy, Ad)) = 0 ✓
- $v(\text{Ad}) = T \in \text{eff}$ ✓

Select: drive(Sy, Ad)
Cost: $C = 8 + 2 = 10$
Preconditions: {at = Sy}
In initial state, done.

Layer 1: Goal at = Ad

This fact is at level 1, and we already selected drive(Sy, Ad) above which achieves it. But let's be explicit:
The action drive(Sy, Ad) also achieves at = Ad as an effect.
No additional cost needed as we already counted this action.

4.6 Final Result

FF Heuristic Value

$h_{FF}(s) = 10$
The relaxed plan:
1. drive(Sy, Br) - cost 1
2. drive(Sy, Ad) - cost 2
3. drive(Ad, Pe) - cost 3
4. drive(Ad, Da) - cost 4

Total: $1 + 2 + 3 + 4 = 10$

Comparison with h+:
The lecture states $h_{FF}(s) = 10 = h^+(s)$. Again, FF found an optimal relaxed plan!

Reality Check:
In the real (unrelaxed) problem, we can't be in multiple places at once. A feasible plan might be:
1. Sy → Ad (cost 2)
2. Ad → Pe (cost 3)
3. Pe → Ad (cost 3)
4. Ad → Da (cost 4)
5. Da → ... (return to Sy visiting Br)

The real optimal cost would be higher than 10 due to having to backtrack, but the relaxed version allows "impossible" plans.

5 Why FF is Not Admissible

5.1 Understanding Inadmissibility

Definition

A heuristic h is **admissible** if $h(s) \leq h^*(s)$ for all states s, where $h^*(s)$ is the true optimal cost.
FF is **not admissible** because it can overestimate the optimal cost.

Wait, but doesn't FF solve a relaxed problem?
Yes! And you might think: "relaxed problem → easier → cheaper solution → underestimate".
Key Insight:

Critical Point

FF finds *some* solution to the relaxed problem, not necessarily the *optimal* solution to the relaxed problem.
$h^+ \leq h^*$ (admissible) because h^+ is optimal relaxed cost.
But h_{FF} might not equal h^+. If $h_{FF} > h^+$, it's still possible that $h_{FF} > h^*$!

5.2 Conceptual Explanation

The two-phase approach of FF:
1. Forward pass: finds when facts first become available
2. Backward pass: greedily selects actions to achieve goals

The backward pass makes **arbitrary choices** when selecting actions. Different choices lead to different relaxed plans with different costs.

Example Scenario:
Suppose:
- Goal g can be achieved by action a_1 (cost 1) or a_2 (cost 10)
- Both a_1 and a_2 are applicable at the same layer
- FF picks a_2 arbitrarily

Result: FF computes cost 10, but optimal relaxed plan uses a_1 with cost 1.
So $h_{FF}(s) = 10 > h^+(s) = 1$.
If the real optimal cost happens to be, say, $h^*(s) = 5$, then:

$$h_{FF}(s) = 10 > h^*(s) = 5$$

This violates admissibility!

5.3 Why Use FF Then?

Despite being inadmissible:
- **Efficient**: Polynomial time computation
- **Informative**: Usually gives good estimates
- **Practical**: Works well with GBFS (Greedy Best-First Search)
- **Balance**: Good tradeoff between accuracy and speed

For optimal planning with A*, we need admissible heuristics. But for satisficing planning (finding any solution, not necessarily optimal), FF works great!

6 Alternative Delete-Relaxation Heuristics

6.1 Overview

Besides FF, there are other ways to extract heuristic values from the delete relaxation:

1. h^+ - Optimal relaxed plan cost (NP-hard, impractical)
2. h_{FF} - Fast Forward heuristic (polynomial, inadmissible)
3. h_{max} - Max heuristic (polynomial, admissible)
4. h_{add} - Additive heuristic (polynomial, inadmissible)
5. h_{LM-Cut} - Landmark-cut heuristic (polynomial, admissible)

6.2 The hmax Heuristic

Idea: Approximate the cost of achieving the most expensive subgoal.

hmax Definition

For a state s and goal set g:

$$h_{max}(s,g) = \begin{cases} 0 & \text{if } g \subseteq s \\ \min_{a \in \mathcal{A}, p \in \text{eff}_a} [1 + h_{max}(s, \text{pre}_a)] & \text{if } g = \{p\} \\ \max_{p \in g} h_{max}(s, \{p\}) & \text{if } |g| > 1 \end{cases}$$

Interpretation:

- If goal already achieved: cost 0
- For single fact p: find cheapest action achieving p, recurse on preconditions
- For multiple facts: take maximum over individual costs

Properties:

- Admissible: $h_{max} \leq h^*$
- Very uninformative: Typically vastly underestimates
- Polynomial time to compute

Why admissible?

h_{max} focuses on the single most costly subgoal, ignoring interactions. This gives a lower bound on the true cost.

Why uninformative?

Consider needing to achieve 10 independent facts, each requiring cost 1. The real cost is 10 (one action per fact), but h_{max} only sees the max of individual costs = 1. So $h_{max} = 1$ while $h^* = 10$.

6.3 The hadd Heuristic

Idea: Sum the costs of achieving individual subgoals.

hadd Definition

For a state s and goal set g:

$$h_{add}(s,g) = \begin{cases} 0 & \text{if } g \subseteq s \\ \min_{a \in \mathcal{A}, p \in \text{eff}_a} [1 + h_{add}(s, \text{pre}_a)] & \text{if } g = \{p\} \\ \sum_{p \in g} h_{add}(s, \{p\}) & \text{if } |g| > 1 \end{cases}$$

Interpretation:

- Same as h_{max} for single facts
- For multiple facts: sum individual costs instead of taking max

Properties:

- Not admissible: Can overestimate
- Can be informative in some domains
- Polynomial time to compute
- Prone to over-counting shared preconditions

Why not admissible?

h_{add} adds costs independently, which can count the same action multiple times if it achieves multiple subgoals.

Example:

Action a achieves both p and q with cost 1.

Goal: $g = \{p, q\}$

- $h_{add}(s, \{p\}) = 1$ (use action a)
- $h_{add}(s, \{q\}) = 1$ (use action a)
- $h_{add}(s, \{p, q\}) = 1 + 1 = 2$

But the true cost is only 1 (one action achieves both)! So $h_{add} = 2 > h^* = 1$.

6.4 Comparison: FF vs hadd

Both are inadmissible, but:

FF vs hadd

FF generally works better because:

- FF constructs an actual plan, respecting some dependencies
- hadd blindly sums costs, double-counting shared actions
- Empirically, FF is more accurate in practice

6.5 The LM-Cut Heuristic

Idea: Based on *landmarks* - facts or actions that must be part of any solution.

Landmarks

A **landmark** is a set of actions such that every plan must use at least one action from this set.
Example: To achieve goal "package at D", you must at some point "load the package" - this is a landmark.

LM-Cut Algorithm:

1. Identify landmarks in the delete-relaxed problem
2. Compute lower bounds based on necessary actions
3. "Cut" landmarks to avoid double-counting
4. Iterate to refine estimate

Properties:

- Admissible: Provides lower bound on h^+
- More informative than h_{max}
- More expensive to compute than FF, but still polynomial
- State-of-the-art admissible delete-relaxation heuristic

Relationship:

$$h_{max} \leq h_{LM-Cut} \leq h^+ \leq h^*$$

LM-Cut is the best admissible approximation of h^+ among common heuristics.

6.6 Summary Table

Heuristic	Admissible	Complexity	Quality
h^+	Yes	NP-hard	Perfect (relaxed)
h_{FF}	No	Polynomial	Good
h_{max}	Yes	Polynomial	Weak
h_{add}	No	Polynomial	Variable
h_{LM-Cut}	Yes	Polynomial	Strong

7 Practical Considerations

7.1 When to Use FF

Good for:

- Satisficing planning (any solution acceptable)
- Greedy Best-First Search (GBFS)
- Large state spaces where speed matters
- Domains with moderate action costs

Not ideal for:

- Optimal planning with A* (use LM-Cut instead)
- Domains where inadmissibility causes problems
- Very small problems (overhead not worth it)

7.2 Computational Complexity

Forward Pass:

- At most $|\mathcal{F}|$ layers (one per fact)
- Each layer processes $|\mathcal{A}|$ actions
- Complexity: $O(|\mathcal{F}| \cdot |\mathcal{A}|)$

Backward Pass:

- Process each goal and subgoal once
- At most $|\mathcal{F}|$ subgoals total
- Each requires finding an action: $O(|\mathcal{A}|)$
- Complexity: $O(|\mathcal{F}| \cdot |\mathcal{A}|)$

Total: $O(|\mathcal{F}| \cdot |\mathcal{A}|)$ - polynomial!

Compare to h^+ which is NP-hard.

7.3 Implementation Tips

Practical Tips

1. **Action Selection**: In backward pass, use heuristics to pick actions (e.g., prefer lower-cost actions, actions achieving multiple goals).
2. **Caching**: Cache layer structures if evaluating multiple states with similar structure.
3. **Early Termination**: If computing heuristic for A* and current f-value exceeds incumbent, can terminate early.
4. **Incremental Computation**: In some cases, can update layers incrementally instead of rebuilding from scratch.
5. **Parallel Layers**: Can compute action applicability in parallel for speedup.

7.4 Common Mistakes

Pitfalls to Avoid

1. **Forgetting fact persistence**: In delete relaxation, facts never disappear. Don't remove facts from F_{i+1}.
2. **Wrong layer for actions**: Action level is the first layer where all preconditions are met.
3. **Incorrect goal distribution**: Use the level where fact first appears, not last.
4. **Double-counting actions**: In backward pass, if an action achieves multiple goals at the same layer, only count its cost once.
5. **Ignoring cost**: Some implementations assume unit cost. Make sure to use actual action costs.
6. **Not checking fixpoint**: If layers stop growing and goal not reached, must return ∞.

8 Advanced Topics

8.1 FF Planner

The **FF (Fast Forward) planner** is a complete planning system, not just a heuristic:

- Uses FF heuristic within GBFS or EHC (Enforced Hill Climbing)
- Won several tracks at International Planning Competition
- Includes helpful action pruning techniques
- Very fast in practice on many benchmark domains

Enforced Hill Climbing (EHC):

1. Expand current state
2. Find successor with strictly better heuristic value
3. If found, move there and repeat
4. If stuck (local minimum), do breadth-first search to escape

EHC + FF heuristic = very effective combination!

8.2 Helpful Actions

Idea: During backward pass of FF, we identify actions that were selected for the relaxed plan. These are called **helpful actions**.

Helpful Actions Pruning:

- When expanding a state in search, only consider helpful actions
- Reduces branching factor dramatically
- Empirically very effective
- Can miss optimal solutions, but finds solutions faster

How to identify:

Modify FF backward pass to record actions selected. These actions form a "relaxed plan" and are marked as helpful for the current state.

8.3 Variations of FF

FF-style heuristics:

- **Weighted FF**: Multiply heuristic by weight $w > 1$ for faster (but less optimal) search
- **FF with better action selection**: Use smart heuristics in backward pass

- **Multi-plan FF**: Extract multiple relaxed plans and average/combine their costs

8.4 Integration with Search

Common search algorithms using FF:

1. **GBFS with FF**: Expand state with lowest h_{FF} value. Fast but not optimal.
2. **WA* with FF**: Use $f = g + w \cdot h_{FF}$ with weight $w > 1$. Bounded suboptimality.
3. **EHC with FF**: Enforced hill climbing, escapes local minima. Very fast in practice.

Why not A* with FF?

A* requires admissible heuristic for optimality guarantee. Since FF is inadmissible, A* with FF doesn't guarantee optimal solutions.

For optimal planning, use A* with h_{LM-Cut} or other admissible heuristics.

9 Connections to Other Concepts

9.1 Planning Graphs

The FF forward pass is essentially building a **planning graph**:

- Alternating fact layers and action layers
- Facts persist across layers (like in Graphplan with no deletes)
- Mutual exclusion (mutex) constraints not used in basic FF

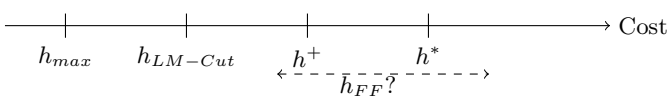
Graphplan is a classical planner that:

1. Builds planning graph forward
2. Searches backward for a valid plan
3. Uses mutex constraints to prune invalid combinations

FF borrows the planning graph structure but:

- Works with delete relaxation (no negative effects)
- Uses it for heuristic estimation, not direct planning
- Doesn't track mutexes (for efficiency)

9.2 Relationship to h+ and h*



h_{FF} can fall anywhere! Sometimes $h_{FF} < h^+$, sometimes $h_{FF} = h^+$, sometimes $h_{FF} > h^+$.

In our examples, $h_{FF} = h^+$, but this is not always the case.

9.3 Delete Relaxation in General

Key insight of delete relaxation:

By ignoring negative effects, we make the problem monotonic:

- States only "grow" (gain facts)
- Progress is irreversible
- Subgoals can be achieved independently (in relaxation)

This is why delete-relaxation heuristics work well: they exploit the "easy" structure of monotonic problems while still being relevant to the original problem.

9.4 Domain-Independent Planning

FF and other delete-relaxation heuristics are **domain-independent**:

- Work for any STRIPS/FDR planning problem
- Don't require domain-specific knowledge
- Automatically extract structure from problem definition

This is a major advantage over domain-specific heuristics which must be hand-crafted for each domain.

10 Worked Practice Problems

10.1 Practice Problem 1: Simple Blocks World

Domain: Blocks world with 3 blocks A, B, C.

Initial State:

- on(A, Table), on(B, Table), on(C, A), clear(B), clear(C), handempty

Goal: on(B, C), on(A, B)

Actions:

- pickup(x): pre: on(x, Table), clear(x), handempty; eff: holding(x), not on(x, Table), not handempty, not clear(x)
- putdown(x): pre: holding(x); eff: on(x, Table), clear(x), handempty, not holding(x)
- stack(x, y): pre: holding(x), clear(y); eff: on(x, y), clear(x), handempty, not holding(x), not clear(y)
- unstack(x, y): pre: on(x, y), clear(x), handempty; eff: holding(x), clear(y), not on(x, y), not handempty, not clear(x)

All unit cost.

Solution - Forward Pass:

$F_0 = \{\text{on}(A, \text{Table}), \text{on}(B, \text{Table}), \text{on}(C, A), \text{clear}(B), \text{clear}(C), \text{handempty}$

A_0:

- pickup(B): clear(B), on(B, Table), handempty all in F_0 ✓
- unstack(C, A): on(C, A), clear(C), handempty all in F_0 ✓

$A_0 = \{\text{pickup}(B), \text{unstack}(C, A)\}$

$F_1 = F_0 \cup \{\text{holding}(B), \text{holding}(C), \text{clear}(A)\}$

(And negative effects are ignored in relaxation, so all original facts persist)

A_1: Now we can stack or putdown:

- stack(B, C): holding(B), clear(C)? Both in F_1 ✓
- stack(C, B): holding(C), clear(B)? Both in F_1 ✓
- stack(B, A): holding(B), clear(A)? Both in F_1 ✓
- And more...

Continue until goals on(B, C) and on(A, B) achieved...

[Full solution left as detailed exercise - requires multiple layers]

10.2 Practice Problem 2: Simple Grid World

Domain: Robot navigation on 2x2 grid.

Locations: (0,0), (0,1), (1,0), (1,1)

Initial: robot at (0,0), goal at (1,1)

Actions: move up/down/left/right (unit cost, preconditions based on grid boundaries)

Task: Compute h_{FF} for initial state.

Solution:
$F_0 = \{\text{at}(0,0)\}$
$A_0 = \{\text{move-right}, \text{move-up}\}$
$F_1 = F_0 \cup \{\text{at}(1,0), \text{at}(0,1)\}$
$A_1 = \{\text{move-up from (1,0)}, \text{move-right from (0,1)}\}$
$F_2 = F_1 \cup \{\text{at}(1,1)\}$ - goal achieved!
Backward pass:

- Goal at(1,1) at level 2
- Select move-right from (0,1) (or move-up from (1,0)), cost 1
- Subgoal: at(0,1) at level 1
- Select move-up from (0,0), cost 1
- Subgoal: at(0,0) at level 0 - done!

$h_{FF} = 1 + 1 = 2$ (Manhattan distance)

11 Key Takeaways

Summary

1. **FF is a two-phase algorithm**: Forward layer building + backward plan extraction.
2. **Delete relaxation** makes the problem easier by ignoring negative effects. Facts persist once achieved.
3. **FF is inadmissible** because it finds a suboptimal relaxed plan. h_{FF} may exceed h^*.
4. **FF is efficient**: Polynomial time, much faster than computing optimal h^+.
5. **FF works well in practice** for satisficing planning with GBFS or EHC.
6. **Alternative heuristics** include h_{max} (admissible, weak), h_{add} (inadmissible), and h_{LM-Cut} (admissible, strong).
7. **Action selection** in backward pass is arbitrary - different choices yield different costs.
8. **Planning graphs** structure is reused from Graphplan, adapted for heuristic computation.
9. **Domain-independent**: FF works on any STRIPS/FDR planning problem without modification.
10. **Helpful actions** pruning, based on relaxed plan, makes search more efficient.

12 Further Reading

Original FF Paper:

- Hoffmann, J., & Nebel, B. (2001). "The FF planning system: Fast plan generation through heuristic search". JAIR, 14, 253-302.

Delete Relaxation:

- Bonet, B., & Geffner, H. (2001). "Planning as heuristic search". AIJ, 129(1-2), 5-33.

LM-Cut Heuristic:

- Helmert, M., & Domshlak, C. (2009). "Landmarks, critical paths and abstractions: What's the difference anyway?". ICAPS 2009.

Planning Graphs:

- Blum, A., & Furst, M. (1997). "Fast planning through planning graph analysis". AIJ, 90(1-2), 281-300.

13 Exam Preparation

13.1 Common Exam Questions

Type 1: Algorithm Tracing

Given a planning problem, trace the FF algorithm:

1. Build fact and action layers
2. Record levels
3. Distribute goals
4. Perform backward extraction
5. Compute heuristic value

Type 2: Theoretical Questions

- Is FF admissible? Why or why not?
- Compare FF with h_{max}, h_{add}, h^+
- Explain delete relaxation
- Why is FF polynomial but h^+ is NP-hard?

Type 3: Modifications

- How would you modify FF for action costs?
- What if we use max instead of sum in backward pass?
- How to extract helpful actions from FF?

13.2 Quick Reference

FF Algorithm Structure:

1. Forward: Build $F_0, A_0, F_1, A_1, \ldots$ until $G \subseteq F_m$
2. Mark: Record level(f) and level(a)
3. Distribute: Create G_t sets
4. Backward: Select actions, accumulate cost, add subgoals
5. Return: Total cost C

Key Formulas:

- $A_m = \{a \in \mathcal{A} \mid \text{pre}_a \subseteq F_m\}$
- $F_{m+1} = F_m \cup \bigcup_{a \in A_m} \text{eff}_a$
- $\text{level}(f) = \min\{t \mid f \in F_t\}$
- $\text{level}(a) = \min\{t \mid a \in A_t\}$

Properties Checklist:

- ☐ FF is inadmissible
- ☐ FF is polynomial time
- ☐ FF uses delete relaxation
- ☐ FF builds planning graph forward
- ☐ FF extracts plan backward
- ☐ Action selection is arbitrary
- ☐ Facts persist in relaxation

14 Conclusion

The FF heuristic represents a elegant balance between computational efficiency and heuristic accuracy. By leveraging the delete relaxation and a two-phase graph-based approach, it computes informative heuristic values in polynomial time, making it practical for large planning problems.

While not admissible, FF's performance in practice, especially when combined with GBFS or EHC, has made it one of the most successful approaches in automated planning. Understanding FF provides insight into:

- Heuristic design principles
- Problem relaxation techniques
- Planning graph structures
- Tradeoffs between optimality and efficiency

These concepts extend beyond FF to many other heuristic search and planning techniques, making FF a cornerstone of modern AI planning.

End of FF Heuristic Cheat Sheet

Machine Intelligence - Lecture 4